Lichen's reference L2

as built and measured

draft

waxwing

Version 0.1, November 2026

a companion to *Lichen*, version 0.5 [1]

Abstract

Lichen [1] enforces an L2's withdrawals through bilateral contracts between a hub and each user, settled on Bitcoin by BABE: a user claims, and only the hub proves, if it can, that her claim is false. This companion describes the L2 that lichen builds to show it, in which the hub is also the sequencer, so that everything a separate L2 would vouch for must be enforced against the operator. Its ledger is one of notes, transparent and shielded, in which every block is proved valid by a chain of RISC Zero proofs and the history is anchored on Bitcoin by a chain of single-use seals. A spend counts against its sender only once she has acknowledged it against an anchor, with its data in hand, so withholding data stalls payments but cannot take funds. An exit claims a definite set of notes, judged at the last anchor a margin before the claim; the hub's statement refutes it on eight grounds, all shown from data the operator holds. We give the design and measurements from a deployment on Mutinynet (November 2026) at real parameters: an honest exit is two transactions and 2.7 kvB with no prover on the user's side; a stale claim was proved false on the hub's 16-vCPU server in about 9 minutes and wrapped in 2, and in 3.5 s on a rented GPU; a payment is firm in 10 to 20 minutes on Mutinynet, about an hour on mainnet. The security argument is a sketch, and the hub's proving capacity against a wave of false claims is the main open question.

Contents

1 Scope **2**

2 When the hub is the sequencer **3**

3 The ledger **3**

4 Validity **4**

5 Finality **5**

6 Payments, and the sender's acknowledgment **5**

6.1 The problem 5

6.2 The rules 6

6.3 Who is protected 6

6.4 What it costs . . . 6

7 The claim **7**

8 The hub's statement **8**

9 Shielded notes **9**

10 Withdrawals **10**

11 Measurements **10**
11.1 Opening . . . 10
11.2 The exits . . . 10
11.3 Regtest . . . 11

12 What the earlier design had, and this one doesn't **12**

13 Precedents **12**

14 Limits and open questions **13**
14.1 The hub's proving capacity . . . 13
14.2 Block space . . . 13
14.3 Data availability . . . 13
14.4 The rest . . . 13

1. Scope

The main paper [1] describes a contract between a user, Alice, and a hub: a 2-of-2 output holding her deposit D, the hub's liquidity up to a cap $V_{\max}$, and two reserves r_A, r_H. The hub's tokens on an L2 are collateralised by it. A withdrawal is a claim C of b, folded cooperatively, or, if the hub refuses or is offline, posted on Bitcoin in `request`, where the hub must prove it false or let a timeout pay her. The hub's statement is the L2's:

$$\mathcal{R}_H\big(\mathcal{D}, c, \ldots;\ w = (\mathsf{req}, \rho, \ldots)\big) \ = \ \neg\,\mathsf{Valid}_L\big(C(\mathsf{req}),\, b(\mathsf{req})\big),$$

with ρ a refutation. How BABE checks the hub's proof on Bitcoin, with the roles swapped so that Alice garbles and the hub proves, and the setup by verifiable secret sharing, are in the main paper's appendix [1, App. A]. This companion is the other side of the interface: lichen's own L2 [3], what Valid_L and its refutations are for it, and how it all measured.

It replaces *Lichen as built* [2], whose two parts described an earlier design: Alice proved her own withdrawal, the hub garbled, a withdrawal began with a return of tokens to the hub, and an exit could be answered only with a counter that Alice might then have to prove false. All of that is gone (Section 12).

Figures marked (lichen) are measured: on Mutinynet, a signet with 30-second blocks, on one deployment opened on 2026-11-11 with seven contracts at real parameters, or on regtest where stated. The rest are estimates, marked (est.). The machines are a 14-core Apple M4 Pro laptop (RISC Zero has no Metal prover, so its CPU); *lichen-hub*, the deployment's own server, 16 x86

vCPUs at 2.0 GHz without AVX; a 4-vCPU x86 server; and a rented NVIDIA RTX 4090. A walk-through of every Mutinynet run, transaction by transaction, is in lichen's explainers [4].

2. When the hub is the sequencer

When the L2 is not the hub's (an L2 secured by its own consensus, or a rollup elsewhere with its own proofs), the L2's account of its ledger is an input, and the hub only a counterparty. In lichen's L2 the hub and the sequencer are one operator, or two that trust each other. That is a design choice, and its consequence is that the L2's account is the operator's own word. A hub profits from defaults, since whatever isn't withdrawn by T_{close} is its, so it has every reason to bend that account, and the hub's statement must not take it on trust.

With hub and sequencer one party, the threat is the operator against its users. Users against users is the ledger's ordinary job, and the operator against itself is out of scope. Table 1 lists what the operator could do, and what answers it.

the operator	answered by
mints notes nobody paid for, and uses them to refute a claim	validity: the chain proof (Section 4)
signs two valid histories that split	one anchored history: the seal chain (Section 5)
anchors a history on a branch of Bitcoin it mined	the contract's work W (Section 5)
anchors a spend of Alice's and withholds its data	sender acknowledgment: a spend counts only once she acknowledges it, with her data (Section 6)
holds a payment of hers unsequenced, then includes it once she claims its inputs	the margin: a claim is judged at an anchor fixed before it (Section 7)
refuses the fold, censors her, stops sealing, or vanishes	the exit: her claim, which the hub must prove false (Sections 7–8)
claims her notes through a contract of its own	only her keys sign a claim of her notes

Table 1: What a merged operator could do, and what answers it.

3. The ledger

The ledger is one of notes. A *transparent* note is an owner key, a value and a memo, with a random ρ; a *shielded* note is a commitment to its owner, value, ρ and a trapdoor, opened only to its owner [5]. Their commitments form one append-only Merkle tree, so a note's path under a root stays valid however the ledger grows. A leaf's top bit records its kind, set by the chain statement as it appends, so that a claim can name a note by its position alone.

A spent note is not removed but recorded by its *nullifier* in the *spent set*, a compact sparse Merkle tree (a key's path is as deep as its keys' common prefixes: some twenty compressions an access at a million keys, not 256 levels). The same tree holds, under domain-separated keys, the state that acknowledgment needs (Section 6): each pending spend, each locked output, each

acknowledgment and each voided spend, and each recorded anchor's history. Nothing is ever deleted. A nullifier's leaf names how the note went: *spent* by a transaction, named by its digest, or *exited* through contract c, by an exit record. A state root hashes the height, the commitment tree's root and the spent set's root, and the sequencer signs it (BIP340).

The transaction types are a mint (the hub's), a transfer, a shielded spend, an acknowledgment, an anchor record (Section 5) and an exit record (Section 10). Three proofs are involved, all RISC Zero (Table 2). No user proves anything for a withdrawal or an exit. A user proves only her shielded spends, and only on a deployment that has them.

proof	made by, when	shows; cost (lichen)
spend	the sender, for each shielded transaction	its notes exist, are hers and are unspent, and the amounts balance; a shield 14.6k cycles, 3.0 s on the laptop; two inputs and two outputs 2^{17} to 2^{20} cycles, 23 to 110 s
chain	the sequencer's prover, for each sealed height	this block is a valid transition, and so was the history before it; 98k cycles (an empty seal) to 0.84M (a payment and its acknowledgment), one segment
the hub's statement	the hub, only for a false claim	the claim posted in `request` is false (Section 8); 1.0 to 1.65M cycles, two segments; wrapped in Groth16 for BABE

Table 2: The three proofs. The guests are built reproducibly in RISC Zero's builder image; a contract's statement names the hub's guest's image id, which names the chain guest's.

4. Validity

The chain guest proves one sealed height: the block's transactions applied to the previous state give the new one; its spends are authorised (a signature, or a verified spend proof for a shielded note), transparent inputs unlocked, no nullifier spent twice, values balanced, mints signed by the hub. It takes the previous height's chain receipt as an assumption, so each receipt vouches for the whole history from genesis. Its journal carries the height, the state root, the anchoring state and a history hash

$$hist_0 = 0, \qquad hist_h = H(\texttt{"lichen/hist"} \,\|\, hist_{h-1} \,\|\, SR_h),$$

which commits the whole sequence of roots to h.

A BIP340 signature, about 260k cycles, is most of a block's work; the spent set, after it was made compact, about a tenth (lichen). One machine proving height after height keeps up with a steady load, but not a burst: on lichen-hub a height took 38 to 157 s, and after a burst of heights in the trial the prover ran 31 to 35 minutes behind (lichen). An honest sequencer's lag is only a delay, but the hub's proof waits for the chain receipt at its claim's state, so the lag counts against the hub's window (Section 14.1). On the RTX 4090 a height proved in 1.4 to 2.3 s, against 18 to 62 s on the laptop (lichen).

A valid history is not a unique one: the sequencer can sign two histories that split, each valid, which the chain proof cannot see. The wallet and the hub record every root they are shown and

refuse to go on when shown a second root at a height, with the evidence. Anchoring makes such a split useless.

5. Finality

The seal chain. The deployment's genesis names an outpoint, the first *seal*, a P2TR output of the sequencer's key. An *anchor* spends the current seal and creates the next, beside an output `OP_RETURN "lichen"` $\|$ h $\|$ $hist_h$; it is 269 vB (lichen). Bitcoin lets an output be spent once, so there is one chain of anchors, after Todd's single-use seals [6]. The sequencer anchors at most once a Bitcoin block, and only when something has changed.

Anchors inside the history. Once an anchor is the margin deep on Bitcoin (20 blocks on Mutinynet, 6 on mainnet), the sequencer records it on the L2, and the chain guest checks the record: the transaction is in the block it names, by Merkle path to the header; it spends the seal the journal names, whose script is checked so that no one else could spend it; and it commits a *hist* that is an ancestor of the current one. A history forked from the anchored one can then hold no anchor after the fork.

The contract's work. The chain guest checks no proof of work. The hub's statement does: `request`'s block must have headers on it, each linked and within its own target, whose work sums to at least W, fixed when the contract opens (ten blocks' work at the difficulty of a checkpoint both sides check against their own view of the chain). A forger gains nothing from headers declaring low difficulty: work is summed. This is not a proof of the heaviest chain; it makes a forged `request` or anchor a matter of mining a fork of Bitcoin as long as W. W, the genesis seal, the deployment's keys and the contract's unit make the deployment's identity,

$$\mathcal{D} = H(\texttt{"lichen/contract"} \,\|\, \text{deployment} \,\|\, \text{genesis seal} \,\|\, W \,\|\, u),$$

which the hub's statement's journal carries (Section 8).

6. Payments, and the sender's acknowledgment

6.1. The problem

An exit claim is judged at an anchored state: "these notes are mine and unspent at A^*". The root of each anchor is on Bitcoin, but the data under it isn't. An operator that anchors a busy block and serves none of its data leaves every sender in it unable to choose what to claim:

- *her inputs:* if the anchor holds her payment, they are spent, and a claim of them is false;
- *her outputs:* if it doesn't, they don't exist, and a claim of them is false.

Whatever she can't claim defaults to the hub at T_{close}: the motive. No better accumulator fixes this: a member's proof needs update data proportional to the others' changes [7], which is what an anchor's data is. The fix is an order: inclusion first, which changes nothing yet; then the sender's signature, given only once she holds her data, which makes the spend count.

6.2. The rules

- *Every spend names a one-time key ak,* the sender's, for that spend only (derived from her wallet's key and the spend's first nullifier; for a shielded spend, a public input of its proof). Transactions the hub signs, its mints and exit records, acknowledge themselves.
- *A sealed spend is pending.* It spends its inputs and appends its outputs, but its outputs are *locked*: no one spends them, the sender's change included, until it is acknowledged.
- *The sender acknowledges against an anchor.* Once an anchor at or above the spend's height is recorded, her wallet checks the anchor on Bitcoin, the sequencer's state against the roots it signed, and that the roots from her block to the anchor hash-chain to the $hist_a$ the anchor commits. Then it signs, under ak, $H(\texttt{"lichen/ack"} \,\|\, txid \,\|\, a \,\|\, hist_a)$. Applied by the chain guest (the spend pending, not void, the anchor a recorded with that history, $h \leq a$), it makes the spend count and unlocks its outputs. Naming the anchor rather than the spend's own height keeps seals as frequent as the sequencer likes.
- *An exit can void a pending spend.* An exit record retiring a nullifier that a pending spend spent voids that spend: its outputs locked for good, a later acknowledgment refused. One spent by an acknowledged spend makes the record invalid.
- *The sequencer's policy,* the hub's protection rather than a guest rule: it sequences an acknowledgment only once its anchor is the margin deep, and not if a `request` on Bitcoin claims the spend's inputs. The earlier of an exit and an acknowledgment wins.

6.3. Who is protected

collusion	what they try	what happens
the operator, against Alice	anchor her payment, withhold its data	she never acknowledges; it counts for nothing, her inputs stay claimable. A stall, not a theft
Alice and the operator, against Bob	never sequence her acknowledgment; she claims her inputs	the exit record voids her payment and Bob's note. He never counted it
Alice and Bob, against the hub	she gives the acknowledgment to Bob alone, and claims her inputs	the hub pays her; its exit record voids the payment, so Bob's claim of his note is refuted as voided. The hub pays once
Alice, against the hub	claims her inputs through one contract and her acknowledged change through another	her acknowledgment, posted in the one, is the hub's evidence against the other

A payee who waits for the acknowledgment to be sequenced is safe against any collusion: from then no exit can void the spend. Before that he holds a soft payment, the sequencer's word and then Alice's acknowledgment.

6.4. What it costs

- *Time.* A payment is firm, and spendable onward, once its anchor is the margin deep and the acknowledgment sequenced. On Mutinynet, Bob's payment sent just after an anchor's

broadcast, the worst case of two anchors, was firm in 39 blocks, 20 minutes; Cleo's, sent before one, in 21 blocks, 10 minutes (lichen). On mainnet, about an hour (est.). There are no fast onward transfers in this version.

- *Presence.* The sender is online until the anchor after her payment is the margin deep. If she leaves first, her payment waits, counting for nothing, and she acknowledges on her return.
- *Proving.* About one BIP340 signature in the chain guest; a payment and its acknowledgment are 0.84M cycles, one segment (lichen).
- *On chain.* Nothing in normal operation. An exit claiming a locked note carries its acknowledgment: under 20 vB.

This is the confirmation signature of Plasma [8] and Plasma MVP [9], with three changes: it binds to a history fixed on Bitcoin, $(a, hist_a)$, rather than to a block; it counts by existence and claims carry it, so the operator can't suppress it by censoring; and outputs are locked until it is sequenced, with an exit voiding a spend still unacknowledged, so that a payee who waits is firm (Section 13).

7. The claim

Alice claims a definite set of her notes, and b, at most $V_{\max}$, in the contract's units. The claim is: *these notes are mine, live at A^*, and worth at least b.* A note is live if its commitment is in the tree and its nullifier is not spent. The claim carries:

- each transparent note by its position (4 bytes); each shielded note by its position and opening (value, ρ and trapdoor: 76 bytes), since the hub knows no shielded opening;
- for each claimed note still locked, the acknowledgment of the spend that made it;
- one signature per owner key on the claim's fingerprint

$$E = H(c \,\|\, O_D \,\|\, b \,\|\, \text{positions} \,\|\, H(\text{acknowledgments}) \,\|\, \ldots),$$

  O_D being the contract's deposit coin, which names the contract on Bitcoin; a shielded owner's signature is by its exit key, beside the keys that derive its nullifiers (Section 9).

`request` posts the claim raw in its witness, at most about 1,200 bytes in items of up to 80 bytes, the most Bitcoin Core relays in a tapscript witness item, beside Alice's one-time Winternitz signatures on b and on E. Script cannot check the claim's bytes; the hub's statement checks that they hash to the signed E, so a relayer can't swap them. A claim holds up to 64 notes, 13 of them shielded.

When it is judged: A^* and the margin. A^* is the last anchor confirmed at least a margin m of blocks before `request`'s block. The margin is for censorship. If the operator holds a payment of hers unsequenced, she sees at the tip that it isn't included and claims its inputs; without a margin, the operator could include the payment in an anchor confirming before her `request` and make her claim false. With it, what she saw when she broadcast is what counts, provided `request` confirms within m blocks, which she ensures by fee-bumping. The tree only grows and a spend is permanent, so the one state at A^* answers every question a claim raises, and the hub's proof needs one chain receipt.

What the wallet claims. Its notes in A^*'s tree, unlocked or carrying their acknowledgment, the largest first, up to the cap and the witness's room. A claim spends whole notes and the contract pays once, so value above b is forfeit; to withdraw an exact amount the wallet first pays itself a note of that size. A note newer than A^* isn't in its tree, so once her notes are anchored the wallet waits until that anchor is the margin deep. With the sequencer gone, it claims from the notes it kept as it read the L2.

8. The hub's statement

The claim posted in `request` *is false.* The statement is fixed when the contract opens: the image id of the hub's guest, which names the chain guest's, and the journal $c \,\|\, \mathcal{D} \,\|\, H(O_D \,\|\, m)$. It reads `request` from Bitcoin, by its inclusion in a block with the contract's work on it, and takes b and E from Alice's signatures' digits: Bitcoin checked those signatures in `request`'s script, so her keys needn't be in the journal. It holds on any one of these grounds (Table 3).

ground	shown by
malformed	the claim doesn't parse, doesn't hash to her E, repeats a note, or b exceeds the cap
not hers	a note's owner signed none of the claim's signatures
too much	the notes are worth less than b
not a note	at A^*, the leaf at a note's position isn't its commitment (an empty leaf past the tree's end included)
spent	at A^*, its nullifier is spent by a spend that counts: hub-signed, acknowledged at A^*, or pending with an acknowledgment the hub shows from anywhere (the L2, another claim on Bitcoin) whose anchor confirmed before the cutoff. This contract's own exit record does not count
locked	at A^*, it is an output of a pending spend, and the claim carries no valid acknowledgment of it
voided	at A^*, it is an output of a spend an exit record voided
a second claim	the note's owner signed a claim of it through another contract

Table 3: The grounds on which the hub's statement holds.

A^* is the last. The hub shows A^*'s anchor confirmed by the cutoff, and the transaction spending A^*'s seal confirmed after it. Only the sequencer's key spends the seal, and Bitcoin allows one spend, so this is objective. If nothing has spent the seal yet, the hub makes the sequencer anchor (on Mutinynet it minted itself a sat, so that there was something to seal), which it can always do. The state at A^* comes from the sequencer's chain receipt at the height where A^*'s own anchor record is applied: anchor records are applied in seal order, so every anchor up to A^* is there, each confirmed before the cutoff.

Why the operator can't forge a refutation. The state comes from a chain receipt, so it is valid; it is the anchored history, and the anchor and `request` carry the contract's work. A nullifier enters the spent set only by a spend, which needs the note's keys, or by an exit record, which the chain guest accepts only with the owners' signatures on that claim. A spend counts

against Alice only with her acknowledgment, which she gives only with her data. A second claim needs her signature, and a copied signature names its own contract. So every refutation traces to something the owner did.

The two kinds of honesty. For an honest claimant no refutation exists: her notes are in the tree at A^*, the spends she hasn't acknowledged don't count, and the spends she has acknowledged she has left out of her claim. For an honest hub, every false claim has one: whatever makes the claim false is in the operator's data. The statement's guest is the decision the hub makes natively, in milliseconds, so the hub proves only what it has already found to hold.

Two claims of one note. Requested together through two contracts, each is the other's second claim, and both are asserted false: a double claimer loses both contracts. After deciding a claim holds, the hub keeps looking for a second claim of its notes until a quarter of its window is left.

9. Shielded notes

An exit must claim shielded notes directly: unshielding is an L2 transaction, which needs the sequencer, which an exit can't count on. The hub knows no shielded opening, and to check a note unspent it needs the nullifier, which needs the owner's key. So the keys are split, and a trapdoor r, chosen by the sender and encrypted to the receiver with the value, enters the commitment and the nullifier:

$$\begin{aligned} \mathit{owner} &= H(\texttt{"lichen/owner"} \,\|\, H(\mathit{ask}) \,\|\, \mathit{nk} \,\|\, X), \\ \mathit{cm} &= H(\texttt{"lichen/snote"} \,\|\, \mathit{owner} \,\|\, v \,\|\, \rho \,\|\, r), \\ \mathit{nf} &= H(\texttt{"lichen/snf"} \,\|\, \mathit{nk} \,\|\, \rho \,\|\, r). \end{aligned}$$

ask authorises spends inside the spend proof and is never revealed. A claim reveals $H(\mathit{ask})$, *nk* and X once per owner, and each note's opening; X, a Schnorr key, signs E and nothing else. The hub then checks everything natively, on the same grounds as for transparent notes, and "not hers" becomes: the owner isn't $H(H(\mathit{ask}) \,\|\, \mathit{nk} \,\|\, X)$ of what was posted, or X's signature doesn't verify.

Revealing *nk* lets anyone link the claimed notes' nullifiers to them, and lets a sender who knows a note's ρ and r see when the owner spent it; nobody can spend with what a claim reveals. So after an exit the wallet moves to a fresh shielded address. The trapdoor also made the commitment hiding: before it, anyone with an address could test leaves against $H(\mathit{owner} \,\|\, v \,\|\, \rho)$ for every plausible value v, ρ being public. The alternatives were key-image nullifiers, which put elliptic-curve multiplications in every shielded spend proof (from 2^{17} to about 2^{20} cycles, est.), a cost on every payment for an exit that should be rare; and a proof by Alice, which is what this design removes.

On Mutinynet, Sara shielded 90,000 sats (3.0 s on the laptop), then exited claiming her shielded note and a transparent one: her `request` was 2,194 vB, 59 more than a one-note transparent claim, and `request_timeout` paid her (lichen).

10. Withdrawals

Cooperative. Alice's wallet builds the same claim an exit would post, and the fold from the contract's terms, and sends both to the hub. The hub checks the claim against its own ledger, now rather than at an anchor (each note hers, unspent, unlocked or carrying its acknowledgment, the owners' signatures for this contract, the value at least b), sequences its *exit record*, which retires the notes and, being hub-signed, counts at once, and then signs and broadcasts the fold. There is no wait for an anchor or a proof: if the hub records the claim and won't fold, her exit posts the very same claim, which this contract's own record doesn't refute; if it never recorded it, the notes are live. Either way she is paid, so she needn't know which. On Mutinynet the hub's part took 123 ms and the fold was 205 vB; Olga's withdrawal took 20 minutes end to end, all of it waiting to acknowledge the split that sized her note (lichen).

The exit. If the hub won't fold, or isn't there, she posts the claim in `request`. As soon as the hub sees it, it holds the claimed notes: the sequencer refuses their spends and any acknowledgment of a spend of them. It decides the statement natively. If the claim holds, it asserts nothing, records the exit on the L2, and lets `request_timeout` pay her after w_H. If it is false, it proves and asserts, and her wallet checks the proof natively (about 1.5 ms): a valid one she concedes, an invalid one she challenges. The transactions and BABE's part are in the main paper [1, App. A].

11. Measurements

All on Mutinynet (lichen), seven contracts of $D = 100{,}000$ sats with 100,000 of liquidity: $V_{\max} = 199{,}155$, 255 units of 781 sats, $r_A = 20{,}000$, $r_H = 31{,}816$, windows $w_H = 240$ blocks (2 hours), $\Delta_2 = 2{,}880$ (a day), $\Delta_1 = 240$.

11.1. Opening

on chain	517 vB: the deposit (212) and the funding (305)
Alice's device	78 instances garbled in 9.3 s on the laptop; 10 kept uploaded, 23.9 MiB each, 239 MiB, 65 s on a home connection (some 30 Mbit/s)
the hub	20 to 36 s checking the 68 opened; 239 MiB stored per contract
signatures	Alice's on 17 pre-signed transactions, the hub's on 7, and its 1,016 adaptor pre-signatures
cut-and-choose	$\binom{78}{10}^{-1} = 2^{-40.2}$

The upload is what the swapped roles cost: under the earlier design the hub garbled and Alice downloaded. Six wallets opening at once over one home uplink took about half an hour.

11.2. The exits

Sizes. `request` 2,135 to 2,194 vB, its floor paid out of the contract; `request_timeout` 565 vB; `assert` 7,725 vB; `challenge_assert` 20,717 vB; `wrongly_challenged` 262 vB; `failure` 589 vB. Sweeping the bit outputs, each of which reads Alice's signature on b again, is about 2.3 kvB, paid from her fee budget. The three links' floors, 30,826 sats, and their three 330-sat anchors

contract	the claim	how it ended	on chain
1002	true	the hub found no ground; `request_timeout` paid her after 2 hours	2.7 kvB
1005	true, a shielded note and a transparent one	the same	2.8 kvB
1003	stale: its note paid away and acknowledged	proved false on lichen-hub, asserted 54 minutes after `request`; conceded	9.9 kvB, then `success`
1001	stale, as 1003	asserted; challenged anyway (a trial switch); answered in two blocks	30.8 kvB
1004	true, against a hub restarted hostile	a forged proof asserted; challenged; no instance opened; `failure` paid her	31.2 kvB

Table 4: Five exits on Mutinynet. The other two contracts were withdrawn cooperatively.

make 31,816: exactly r_H, which the hub sizes for the deepest dispute. In contract 1004, $r_A + r_H$ less that left Alice exactly her own reserve, besides her b.

The hub's proving. The hub's statement for a stale claim proved in 520.6 and 526.0 s on lichen-hub (1.62M and 1.65M cycles, two segments) and wrapped in Groth16 in 128.4 and 119.3 s. Of the 54 minutes from `request` to `assert` in contract 1003, 35 waited for the chain prover, behind after a burst of heights, and 11 were proving and wrapping. On regtest the same kind of statement (1.02M cycles) proved in 827 s on the 4-vCPU server and wrapped in 268 s; on the RTX 4090 it proved in 3.5 s (lichen). RISC Zero's GPU Groth16 prover failed on that card each time (an illegal memory access), so the wrap stayed on the CPU, about 4.5 minutes: nearly all of a GPU hub's proving time. With a working GPU wrap the whole proof would be some 10 to 20 s (est.).

Times. A true claim was paid 241 blocks after `request`, about 2 hours, and swept 41 blocks later, with no prover on Alice's side and nothing needed from the hub or the sequencer after her `request`. Against the lying hub, contract 1004 was paid 274 blocks, 2 hours 23 minutes, after its `request`.

11.3. Regtest

The cases not played on Mutinynet ran on regtest, on the mock statement: one note claimed through two contracts at once (both asserted false, the double claimer losing both); an exit after a withdrawal the hub recorded but didn't fold (paid); the hub and sequencer gone (paid by timeout from the notes kept); a payment never acknowledged with its input claimed (paid; the payment voided and the payee's note locked for good); and a payment acknowledged whose acknowledgment the operator withheld (the locked change claimed, carrying the acknowledgment: paid, the payee's note unlocked).

12. What the earlier design had, and this one doesn't

- *Alice's proof,* and her prover: no STARK, no Groth16 wrap, no x86 machine with Docker and 8 GB, for any withdrawal or exit.
- *The counter,* and Alice's worst case: proving a forged counter false meant verifying a Groth16 proof inside RISC Zero, 56.3M cycles, about an hour on a laptop. The hub now carries the only proof, from data it holds.
- *The return,* a payment of b to the hub with the contract's memo, its withdrawal statement and its own dispute; and the `respond` link, in which the hub signed b. A withdrawal is now a claim, the same object cooperatively or on chain.
- *Exit kits* and the exit set's chain receipt: Alice needs her notes' positions and openings, which her wallet has.
- *The keyless tower.* Under the swapped roles a challenge reveals Alice's secrets, so she challenges herself, in a window of a day on Mutinynet (Section 14).

13. Precedents

How other designs treat a hostile operator, and where lichen differs.

	when data is withheld	sender sign-off	what L1 enforces
Plasma [8]	mass exit, coordinated	both parties acknowledge after inclusion	bonds, challenge games
Plasma MVP [9]	exit within 7 days	confirmation signatures	a priority queue
More Viable Plasma [10]	in-flight exits, canonicity games	dropped	youngest-input priority
Plasma revisited, 2023 [11]	forced inclusion, or L1 reverts	none	a queue, reversion
Intmax2 [12]	unsigned counts for nothing	BLS-signs the root before posting	the signer set, balance proofs
lichen	she claims; the hub proves the claim false or pays	signs $(txid, a, hist_a)$; outputs locked until then	nothing about the L2: an anchor is an `OP_RETURN`

1. *Bitcoin enforces nothing about the L2,* so every rule is applied after the fact, in a dispute. The precedents use L1 to stop a bad state or rescue funds; Bitcoin can't reject an anchor, freeze, or revert. Lichen never prevents a bad anchor: the hub's statement decides which anchored facts count against a claim.
2. *Per-user collateral, not a pool.* A Plasma exit draws on one contract, so a double exit steals from everyone, hence priorities and coordination. Here each exit is against one contract's collateral: no queue, no ordering among users.

3. *The burden on the data holder.* Everywhere else the exiter proves something: inclusion, a path, a balance proof. Here she claims and the operator proves, since it holds all the data and pays the exits.

4. *No in-flight exits.* MoreVP's in-flight exit claims "the inputs or the outputs, whichever is canonical", with bonded games of several parties. On Bitcoin each party in a game is another garbled-circuit setup; acknowledgment keeps claims definite and disputes two-party. MoreVP's objections to confirmation signatures still apply and are accepted: the extra step, griefing by withholding (a stall), and the sender's free option until she signs, which the payee closes by waiting.

14. Limits and open questions

14.1. The hub's proving capacity

The hub must prove every false claim within w_H, and each one it fails to answer is paid. A liar loses a contract, so false claims are costly one at a time; but Sybils who open many contracts, pay their tokens to each other and exit together could outrun one prover. One proof per contract is needed, since each journal is its contract's. On lichen-hub a proof took about 11 minutes with its wrap, after the chain prover had caught up; on a GPU the proving is seconds and the wrap the rest. Options: a batched STARK with a small composing guest and a wrap per contract; a working GPU wrap; a longer w_H, which is also the honest user's wait; a limit on contracts opened per window.

14.2. Block space

Each exit is a dispute competing for block space. Against a hub that asserts falsely against everyone, each user's challenge is 20.7 kvB within Δ_2; with N contracts and a window of w blocks, $N \cdot 20.7\,\text{kvB} \leq w \cdot 1\,\text{MvB}$ allows about 7,000 contracts a day-long window on mainnet (est.), more as the window grows. Each false assertion also costs the hub 7.7 kvB and the contract. Against a wave of false claims, the honest hub's 7.7 kvB assertions must fit within w_H.

14.3. Data availability

Acknowledgment closes the data-availability gap for funds: withholding stalls payments but takes nothing. What remains are assumptions: the operator can stop anchoring, or not sequence acknowledgments, so that payments stall; and softer finality (spends of locked outputs that count only if their parents do, as with unconfirmed Bitcoin outputs, or a hub-signed receipt for an acknowledgment) needs analysis before it is built.

14.4. The rest

- *A shielded input's lock* is kept by the wallet and the sequencer, not the chain guest, since its position is private; the guest should enforce it.
- *A tower for the challenge.* A threshold of towers holding Alice's secrets, so that only a quorum colluding with the hub could steal; or accept that challenging is hers, in a window of days.

- *The upload,* 239 MiB per contract: fewer kept instances at larger N, or Duty-Free Bits' smaller encoding [1, App. A].
- *The seal's other spenders.* If the seal gains a freeze path, the first spend of A^*'s seal might be the freeze; the last-anchor argument needs only some spend after the cutoff, but this should be checked.
- *The security argument,* here a sketch: honest-claimant soundness (no refutation of a true claim) and the hub's completeness (a refutation of every false one) written as proofs, beside BABE's own and the cut-and-choose's.

References

[1] waxwing. Lichen: L2s in channels enforced by Bitcoin. Version 0.5, November 2026. https://github.com/AdamISZ/lichen/blob/master/papers/lichen-v0.5.0.pdf

[2] waxwing. Lichen as built: the BABE gadget, the hub's ledger and the exit. Version 0.1 (not maintained), October 2026. https://github.com/AdamISZ/lichen/blob/master/papers/lichen-built-v0.1.1.pdf

[3] lichen: the ledger, the sequencer, the hub and the wallet. https://github.com/AdamISZ/lichen

[4] lichen's explainers: opening a contract, payments and acknowledgments, a cooperative withdrawal, exiting against a hostile operator, the payout, the proving pipeline. November 2026. https://github.com/AdamISZ/lichen/tree/master/docs/explainers

[5] lichen's shielded transfers: an example L2. https://github.com/AdamISZ/lichen/blob/master/docs/SHIELDED.md

[6] P. Todd. Preventing consensus fraud with commitments and single-use-seals, 2016. https://petertodd.org/2016/commitments-and-single-use-seals

[7] P. Camacho and A. Hevia. On the impossibility of batch update for cryptographic accumulators. LATINCRYPT 2010.

[8] J. Poon and V. Buterin. Plasma: scalable autonomous smart contracts, 2017. https://www.plasma.io/plasma.pdf

[9] V. Buterin. Minimal viable Plasma, 2018. https://ethresear.ch/t/minimal-viable-plasma/426

[10] More Viable Plasma, 2018. https://ethresear.ch/t/more-viable-plasma/2160

[11] V. Buterin. Exit games for EVM validiums: the return of Plasma, 2023. https://vitalik.eth.limo/general/2023/11/14/neoplasma.html

[12] E. Rybakken, L. Hioki and M. Yaksetig. Intmax2: a ZK-rollup with minimal onchain data and computation costs featuring decentralized aggregators. ePrint 2023/1082. https://eprint.iacr.org/2023/1082